

Computer Vision

Lecture 2

馬誠佑

CV Projects Ideas

<https://www.projectpro.io/article/computer-vision-projects/437>

1. Cartoonize an Image
2. Face Detection
3. Similar Images Finder
4. Face Recognition
5. Barcode and QR Code Scanner
6. Face Mask Detection
7. Handwritten Character Recognition using MNIST Dataset
8. Number of People Counter
9. Virtual Invigilator
10. Polyp Segmentation
11. Early Fire Detection System
12. Facial Expression Recognition
13. Text Scanner
14. Number-Plate Reader
15. Projects with Open Images Dataset
16. Deep dreams using CNN
17. Image Retrieval using Content
18. Multi-class Image Classification
19. Neural Image Transfer
20. Traffic Sign Classification
21. Waste Segregation System
22. Anomaly Detection in CCTV Footage
23. Autonomous Drone Navigation
24. Social Media Checker
25. Hand Gesture Recognition

<https://grand-challenge.org>

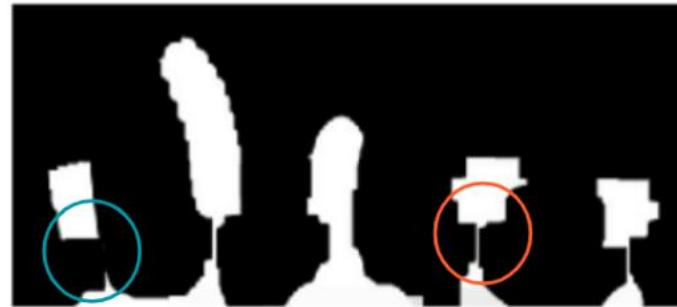
Things you should consider...

- Deep Learning?
- Data ?
- Pretrained model?
- Package?
- API? (chatGPT, stable diffusion, midjourney...)

Erosion



Original Image



Basic kernel



Ellipse kernel



Cross kernel

Dilation



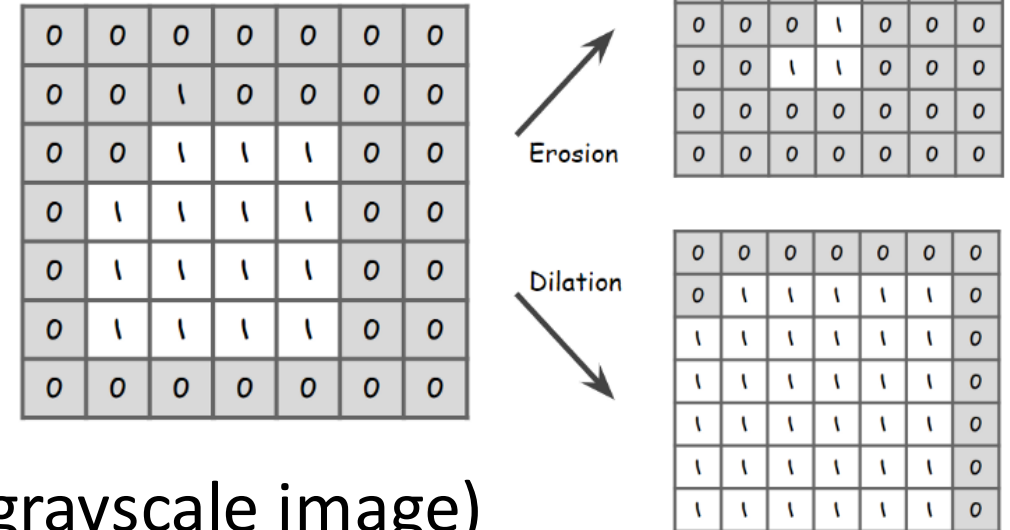
Original Image



dilation

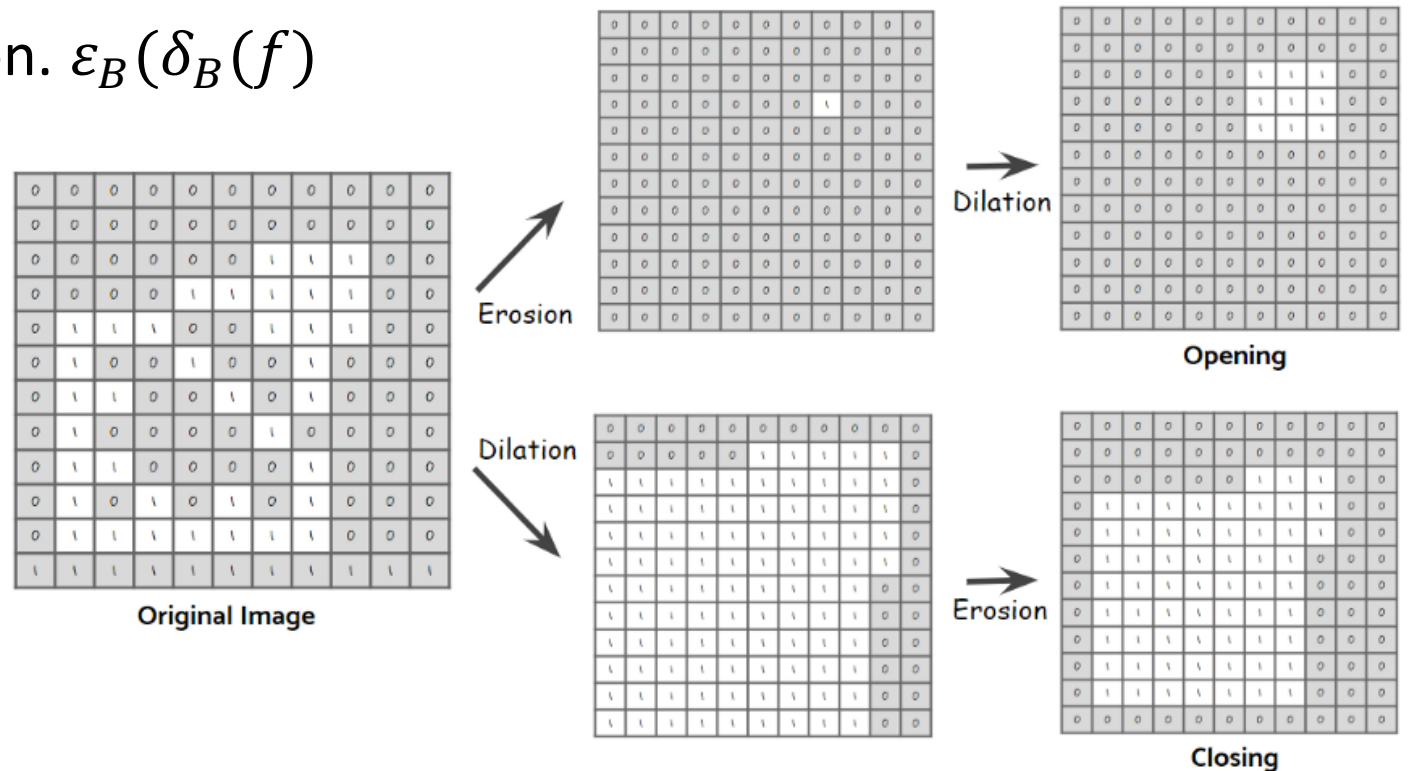
Morphological transformations

- Manipulate image through filter:
 - Rectangle
 - Ellipse
 - Cross shape
 - Erosion: shrinking figures (Usually in grayscale image)
 - Dilation
- | | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |



Morphological transformations

- Opening and closing operation
- mixed version of erosion and dilation
- Opening: First erosion then dilation. $\delta_B(\varepsilon_B(f))$
- Closing: First dilation then erosion. $\varepsilon_B(\delta_B(f))$
- 膨脹 (dilation) : $\delta_B(f)$
- 侵蝕 (erosion) : $\varepsilon_B(f)$



Opening



Closing



Gradient



Top Hat



Black Hat





Original Image



Opening



Closing



Gradient



Tophat



Blackhot

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
# Apply the operations
img = cv2.imread('simpson.jpg')
kernel = np.ones((9, 9), np.uint8)
img_open = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
img_close = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
img_grad = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)
img_tophat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)
img_blackhat = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)

images = [img, img_open, img_close, img_grad, img_tophat, img_blackhat]

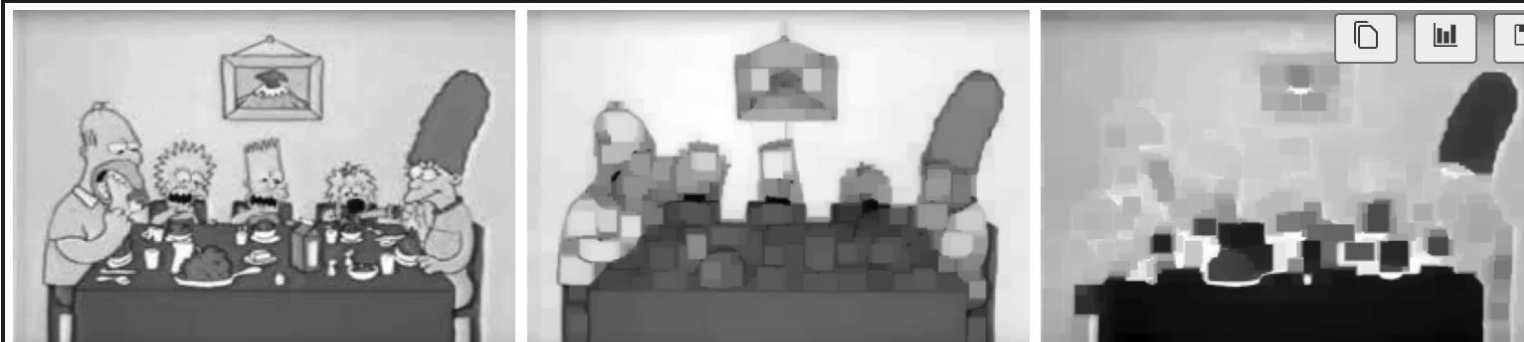
# Plot the images

plt.figure(figsize = (20, 10))
for i in range(6):
    plt.subplot(2, 3, i+1)
    plt.imshow(images[i], cmap = 'gray')
    plt.axis('off')

plt.tight_layout()
plt.show()

```

- **MORPH_OPEN:** Noise Reduction, Noise Reduction, Smoothing and Smaller Object Removal, Background Subtraction. $\gamma_B(f) = \delta_B(\varepsilon_B(f))$
- **MORPH_CLOSE:** Closing Small Holes, Connecting Broken Structures, Removing Small Bright Regions, Text or Character Enhancement. $\varphi_B(f) = \varepsilon_B(\delta_B(f))$
- **MORPH_GRADIENT:** Edge Detection, Object Borders, Shape Analysis, Contour Extraction. $\delta_B(f) - \varepsilon_B(f)$
- **MORPH_TOPHAT:** Enhancing Fine Structures, Detecting Anomalies or Artifacts, Background Subtraction, Medical Imaging, Text Detection, Particle Analysis. $tophat_B(f) = f - \gamma_B(f)$



- **MORPH_BLACKHAT:** Detecting Dark Structures, Anomaly Detection, Background Subtraction, Medical Imaging, Text Detection, Particle Analysis.
 $blackhat_B(f) = \varphi_B(f) - f$

Kernels with different shapes

Rectangular Kernel

```
>>> cv.getStructuringElement(cv.MORPH_RECT, (5,5))  
array([[1, 1, 1, 1, 1],  
       [1, 1, 1, 1, 1],  
       [1, 1, 1, 1, 1],  
       [1, 1, 1, 1, 1],  
       [1, 1, 1, 1, 1]], dtype=uint8)
```

Elliptical Kernel

```
>>> cv.getStructuringElement(cv.MORPH_ELLIPSE, (5,5))  
array([[0, 0, 1, 0, 0],  
       [1, 1, 1, 1, 1],  
       [1, 1, 1, 1, 1],  
       [1, 1, 1, 1, 1],  
       [0, 0, 1, 0, 0]], dtype=uint8)
```

Cross-shaped Kernel

```
>>> cv.getStructuringElement(cv.MORPH_CROSS, (5,5))  
array([[0, 0, 1, 0, 0],  
       [0, 0, 1, 0, 0],  
       [1, 1, 1, 1, 1],  
       [0, 0, 1, 0, 0],  
       [0, 0, 1, 0, 0]], dtype=uint8)
```

Edge Detection

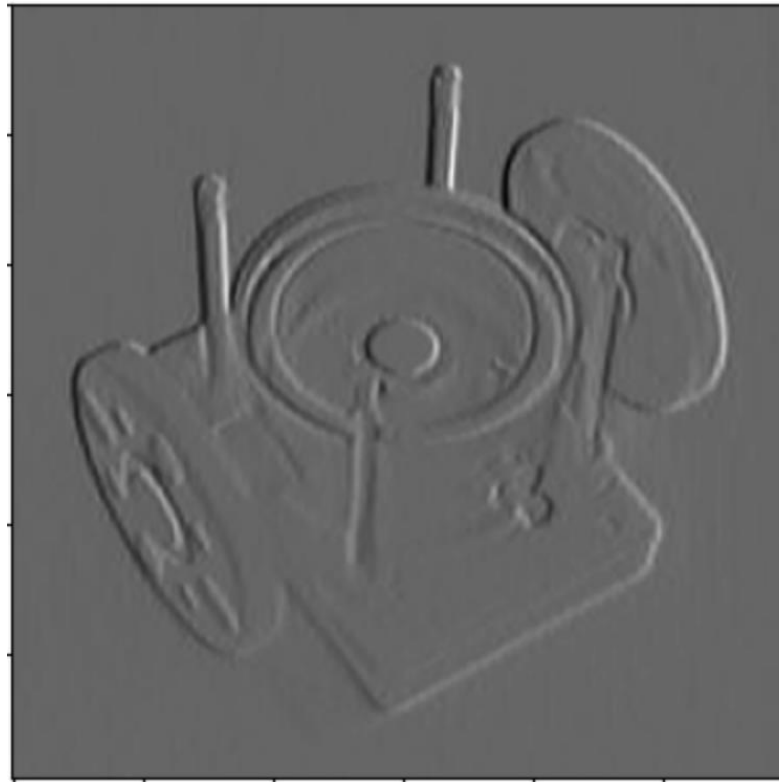
- Sobel operator (`cv2.Sobel()`)
- Canny edge detector(`cv2.Canny()`).

Sobel

- `dst = cv2.Sobel(src, ddepth, dx, dy, ksize=3, scale=1.0)`
- `src`: input image
- `ddepth`: output image depth
- `dx`: order of the derivative x
- `dy`: order of the derivative y
- `ksize`: size of the extended Sobel kernel; it must be 1, 3, 5, or 7
- `scale`: optional scale factor for the computed derivative values

```
ddepth = cv2.CV_64F # 64-bit float output
dx = 1 # First-order derivative in x-direction
dy = 0 # NO derivative in y-direction
sobelx = cv2.Sobel(input_image, ddepth, dx, dy)

plt.imshow(sobelx, cmap='gray')
```



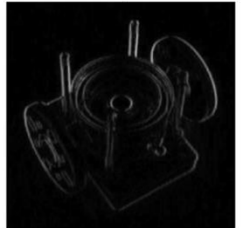
```
sobelx = cv2.Sobel(input_image, cv2.CV_64F, 1, 0)
sobely = cv2.Sobel(input_image, cv2.CV_64F, 0, 1)
sobel = cv2.sqrt(cv2.addWeighted(cv2.pow(sobelx, 2.0), 1.0, cv2.pow(sobely, 2.0), 1.0, 0.0))

plt.figure(figsize=(6, 6));
plt.subplot(2, 2, 1); plt.title('Input image'); plt.axis('off');
plt.imshow(input_image, cmap='gray');
plt.subplot(2, 2, 2); plt.title('Sobel x operation'); plt.axis('off');
plt.imshow(cv2.absdiff(sobelx, 0.0), cmap='gray')
plt.subplot(2, 2, 3); plt.title('Sobel y operation'); plt.axis('off');
plt.imshow(cv2.absdiff(sobely, 0.0), cmap='gray');
plt.subplot(2, 2, 4); plt.title('Sobel intensity'); plt.axis('off');
plt.imshow(sobel, cmap='gray');
```

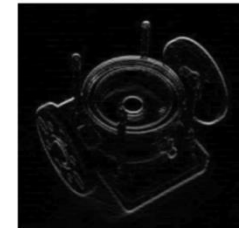
Input image



Sobel x operation



Sobel y operation



Sobel intensity



Canny Edge Detection

```
edges = cv2.Canny(image, threshold1, threshold2, apertureSize=3,  
L2gradient=False)`
```

image: 8-bit grayscale input image

threshold1/threshold2: thresholds for the hysteresis procedure

apertureSize: aperture size for the Sobel() operator

L2gradient: A flag. “True” to use L₂-norm of gradients. L₁-norm for “False”

Process of Canny edge detection

- The process of Canny edge detection algorithm can be broken down to five different steps:

1. Apply [Gaussian filter](#) to smooth the image in order to remove the noise

2. Find the intensity gradients of the image

$$\text{Edge_Gradient } (G) = \sqrt{G_x^2 + G_y^2}$$

$$\text{Angle } (\theta) = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

3. Apply gradient magnitude thresholding or lower bound cut-off suppression to get rid of spurious response to edge detection

4. Apply double threshold to determine potential edges

5. Track edge by [hysteresis thresholding](#): Finalize the detection of edges by suppressing all the other edges that are weak and not connected to strong edges.

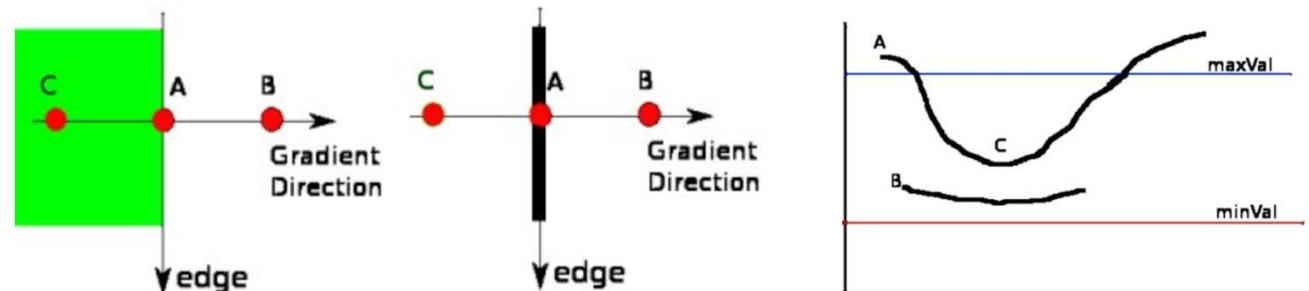
設定兩個閾值：high threshold 與 low threshold。

大於 high → 一定是邊緣。

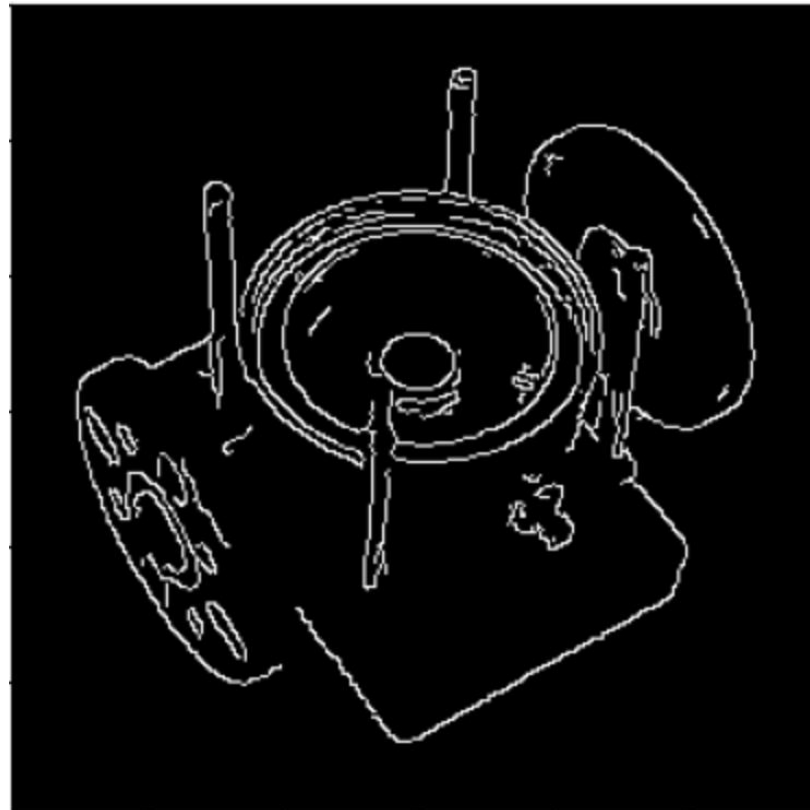
小於 low → 一定不是邊緣。

介於兩者之間 → 只有在「連接到強邊緣」時才算邊緣。

這樣能避免雜訊產生的假邊緣，又不會斷掉真實邊緣。



```
threshold1 = 100  
threshold2 = 200  
canny = cv2.Canny(input_image, threshold1, threshold2)  
  
plt.imshow(canny, cmap='gray')
```



```
# Edge detections with the various thresholds
```

```
canny1 = cv2.Canny(input_image, 50, 150)  
canny2 = cv2.Canny(input_image, 100, 200)  
canny3 = cv2.Canny(input_image, 150, 250)
```

```
plt.figure(figsize=(9, 3));  
plt.subplot(1, 3, 1); plt.title('Canny 50/150'); plt.axis('off');  
plt.imshow(canny1, cmap='gray');  
plt.subplot(1, 3, 2); plt.title('Canny 100/200'); plt.axis('off');  
plt.imshow(canny2, cmap='gray');  
plt.subplot(1, 3, 3); plt.title('Canny 150/250'); plt.axis('off');  
plt.imshow(canny3, cmap='gray');
```

Canny 50/150



Canny 100/200



Canny 150/250



```
img = cv2.imread('Warhammer40k.jpg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

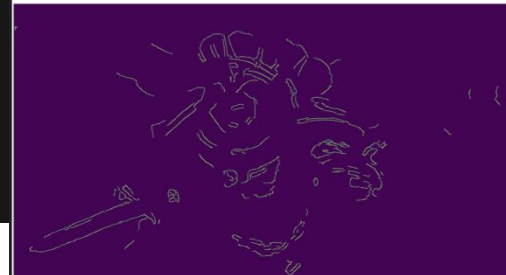
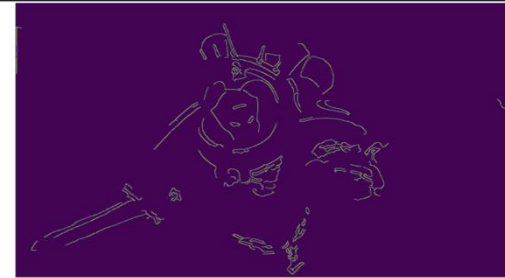
# Canny 檢測 沒有 模糊
# Canny detection without blurring

edges = cv2.Canny(image=img, threshold1=125, threshold2=125)
plt.figure(figsize = (20, 20))
plt.subplot(1, 2, 1); plt.imshow(img)
plt.axis('off')
plt.subplot(1, 2, 2); plt.imshow(edges)
plt.axis('off')
```



```
med_val = np.median(img)
lower = int(max(0, .7*med_val))
upper = int(min(255, 1.3*med_val))

img_k5 = cv2.blur(img, ksize = (5, 5))
# Canny detection with different thresholds
edges_k5 = cv2.Canny(img_k5, threshold1 = lower, threshold2 = upper)
edges_k5_2 = cv2.Canny(img_k5, lower, upper+100)
# Blurring with ksize = 9
img_k9 = cv2.blur(img, ksize = (9, 9))
# Canny detection with different thresholds
edges_k9 = cv2.Canny(img_k9, lower, upper)
edges_k9_2 = cv2.Canny(img_k9, lower, upper+100)
# Plot the images
images = [edges_k5, edges_k5_2, edges_k9, edges_k9_2]
plt.figure(figsize = (80, 50))
for i in range(4):
    plt.subplot(2, 2, i+1)
    plt.imshow(images[i])
    plt.axis('off')
plt.show()
```

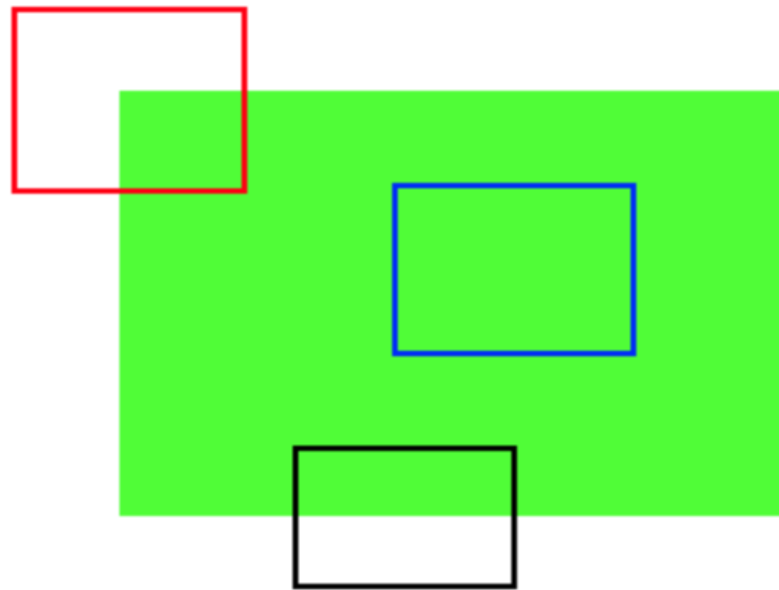
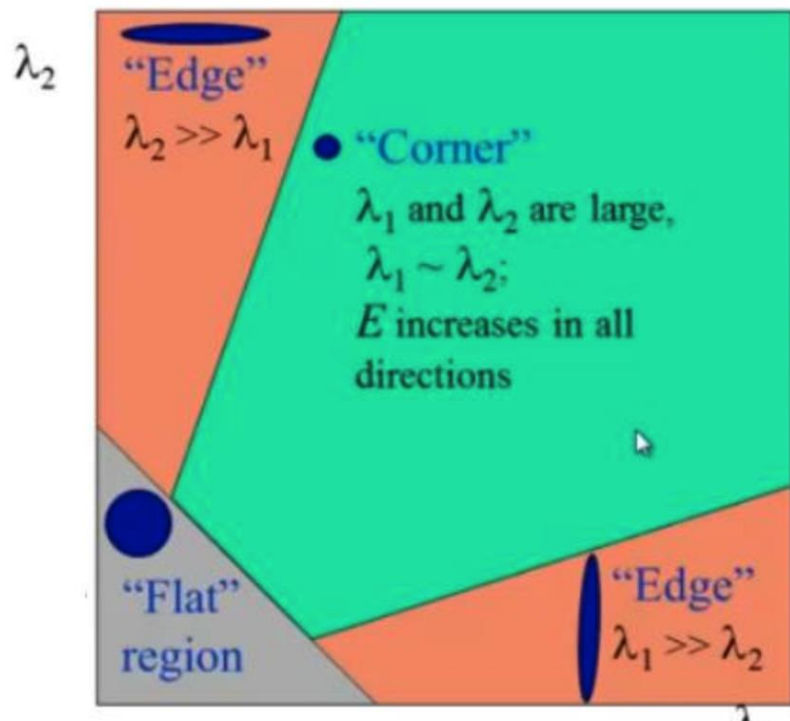


Corner Detection

- Object detection, Motion detection, Video tracking
- Corner: Junction where edges intersect
- Find all edges first, then the intersection point of edges.
- Harris, Shi & Tomasi

Harris Corner Detection

- `cv2.cornerHarris()`



It basically finds the difference in intensity for a displacement of (u, v) in all directions.

$$E(u, v) = \sum_{x,y} \underbrace{w(x, y)}_{\text{window function}} \underbrace{[I(x + u, y + v) - I(x, y)]^2}_{\text{shifted intensity} - \text{intensity}}$$

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \sum_{x,y} w(x, y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$$

where

- When $|R|$ is small, which happens when λ_1 and λ_2 are small, the region is flat.
- When $R < 0$, which happens when $\lambda_1 \gg \lambda_2$ or vice versa, the region is edge.
- When R is large, which happens when λ_1 and λ_2 are large and $\lambda_1 \sim \lambda_2$, the region is a corner.

$$R = \det(M) - k(\text{trace}(M))^2$$

- $\det(M) = \lambda_1 \lambda_2$
- $\text{trace}(M) = \lambda_1 + \lambda_2$
- λ_1 and λ_2 are the eigenvalues of M

```
img = cv2.imread('desk.jpg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
dst = cv2.cornerHarris(img_gray, blockSize = 2, ksize = 3, k = .04) #Apply Harris corner detector
img_corner = img.copy()
img_corner[dst>0.01*dst.max()]=[255,0,0]#coloring the corner

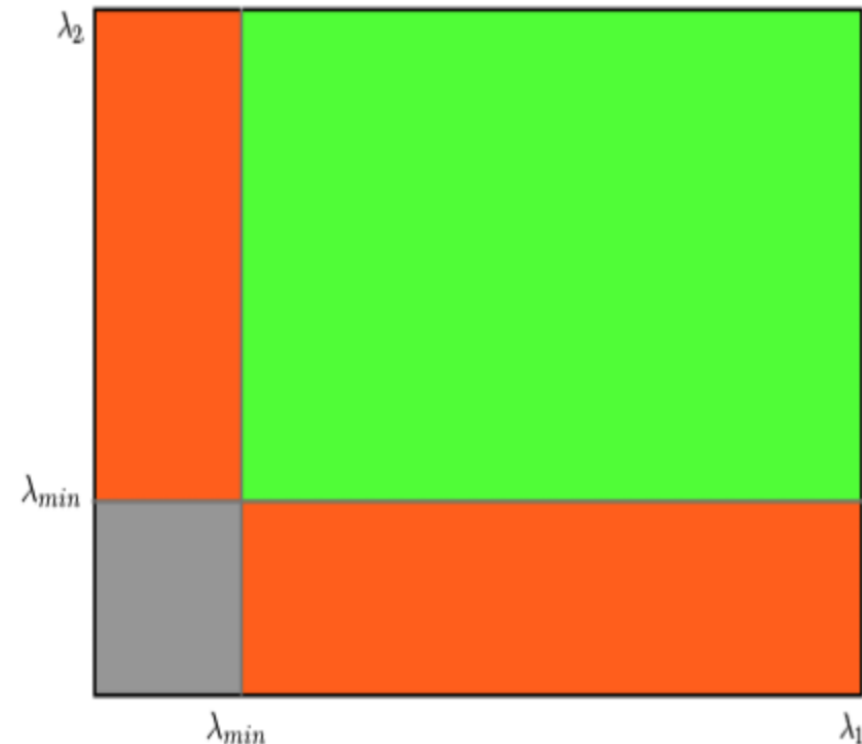
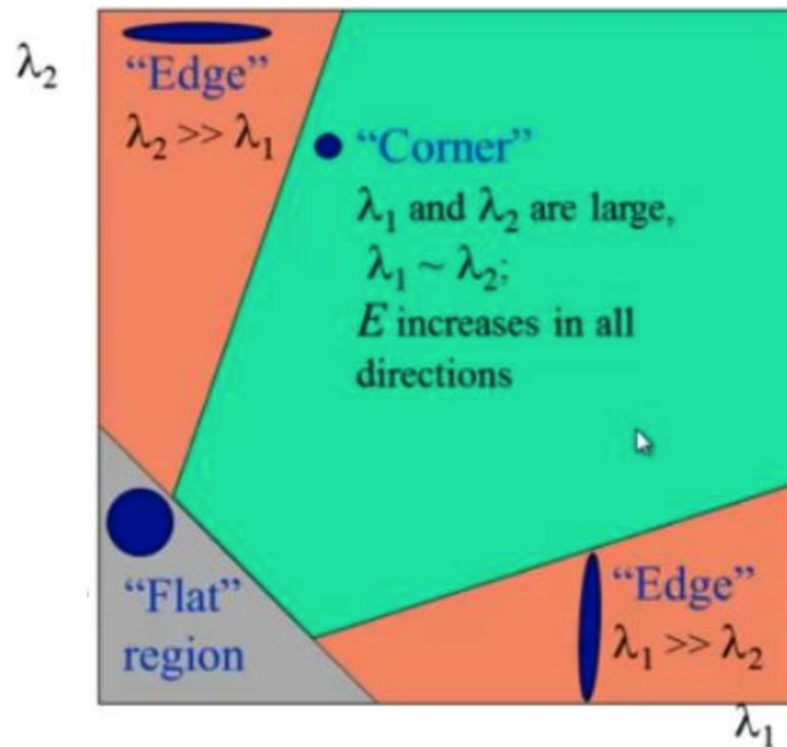
plt.figure(figsize = (20, 20))
plt.subplot(1, 2, 1); plt.imshow(img)
plt.axis('off')
plt.subplot(1, 2, 2); plt.imshow(img_corner)
plt.axis('off')
```



- "src - Input grayscale image, float32 type."
- "blockSize - Neighborhood size used for corner detection, which is the size of the window mentioned above."
- "ksize - Size of the Sobel operator used to compute the gradient image."
- "k - Parameter 'k' used to compute the corner response function, typically in the range of 0.04 to 0.06."

Shi-Tomasi

$$R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2 \longrightarrow R = \min(\lambda_1, \lambda_2)$$



```
corners = cv2.goodFeaturesToTrack(img_gray, maxCorners = 50,  
                                qualityLevel = 0.01,  
                                minDistance = 10)  
corners = np.int0(corners)
```

```
img_croner = img.copy()
for i in corners:
    x,y = i.ravel()

    cv2.circle(img_corner, center = (x, y),
               radius = 2, color = (0,255,0), thickness = -1)
```

```
# Plot the image
plt.figure(figsize = (20, 20))
plt.subplot(1, 2, 1); plt.imshow(img)
plt.axis('off')
plt.subplot(1, 2, 2); plt.imshow(img_corner)
plt.axis('off')
```

image	Input 8-bit or floating-point 32-bit, single-channel image.
corners	Output vector of detected corners.
maxCorners	Maximum number of corners to return. If there are more corners than are found, the strongest of them is returned. <code>maxCorners <= 0</code> implies that no limit on the maximum is set and all detected corners are returned.
qualityLevel	Parameter characterizing the minimal accepted quality of image corners. The parameter value is multiplied by the best corner quality measure, which is the minimal eigenvalue (see cornerMinEigenVal) or the Harris function response (see cornerHarris). The corners with the quality measure less than the product are rejected. For example, if the best corner has the quality measure = 1500, and the <code>qualityLevel=0.01</code> , then all the corners with the quality measure less than 15 are rejected.
minDistance	Minimum possible Euclidean distance between the returned corners.
mask	Optional region of interest. If the image is not empty (it needs to have the type <code>CV_8UC1</code> and the same size as <code>image</code>), it specifies the region in which the corners are detected.
blockSize	Size of an average block for computing a derivative covariation matrix over each pixel neighborhood. See cornerEigenValsAndVecs .
useHarrisDetector	Parameter indicating whether to use a Harris detector (see cornerHarris) or cornerMinEigenVal .
k	Free parameter of the Harris detector.



Face Detection

- “Face Detection” is not “Face Recognition”.
- Cannot identify who that person is.
- Face detection is a classification task
- Haar Algorithm: Haar Feature-based Cascade Classifier (pre-trained model)

Haar Algorithm

- Haar features extraction

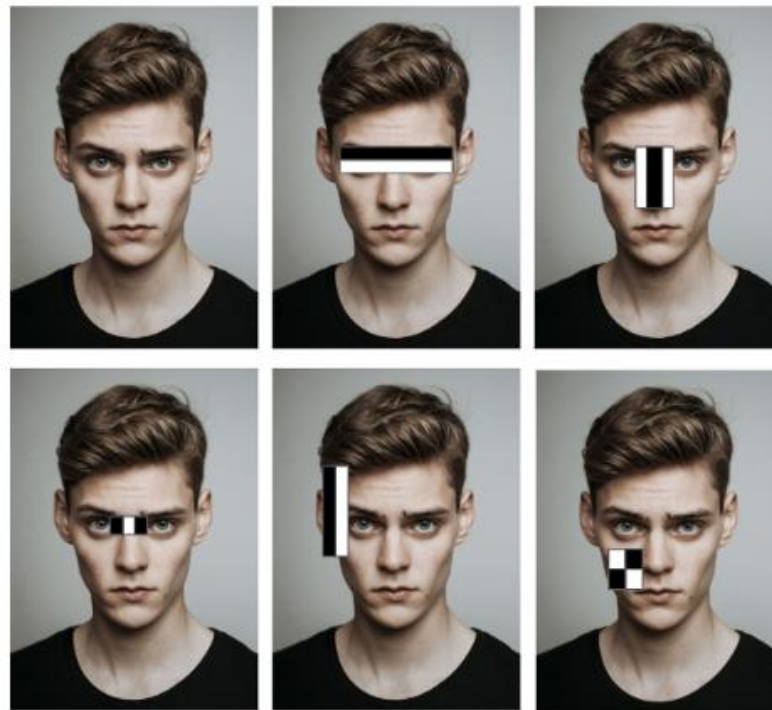
- Integral image

- Adaboost

- Cascade classifier

優點：運算速度快（積分影像 + 級聯結構），當年在即時臉部偵測上非常成功。

缺點：對光照變化、旋轉、遮擋敏感；準確率不如後來的 HOG+SVM、CNN。



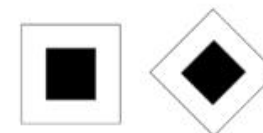
- Edge Features



- Line Features



- Center-surround Features



- Four-rectangle Features



0.6	0.8	0.5	0.6	0.6	0.7
0.6	0.5	0.7	0.7	0.8	0.8
0.1	0.1	0.1	0.2	0.3	0.2
0.2	0.3	0.2	0.3	0.2	0.1

→ $\text{Mean}(\text{dark region}) - \text{Mean}(\text{light region})$

Haar-like features

15	12	11	15	13
28	18	13	14	14
11	15	10	20	42
10	30	17	25	33
21	50	20	17	19

Original Image

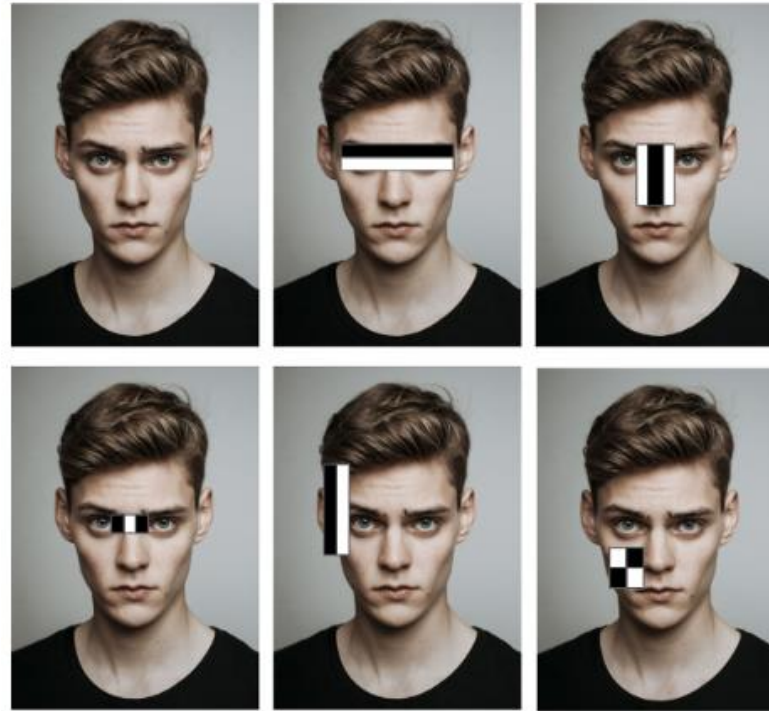


15	27	38	53	66
43	73	97	126	153
54	99	133	182	251
64	139	377	264	366
85	210	281	372	493

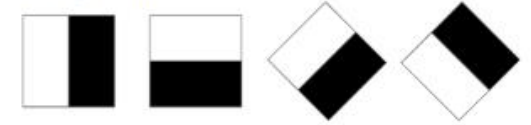
Integral Image

Sum of pixels in orange area

$$= I(D) + I(A) - I(B) - I(C)$$



- Edge Features



- Line Features



- Center-surround Features

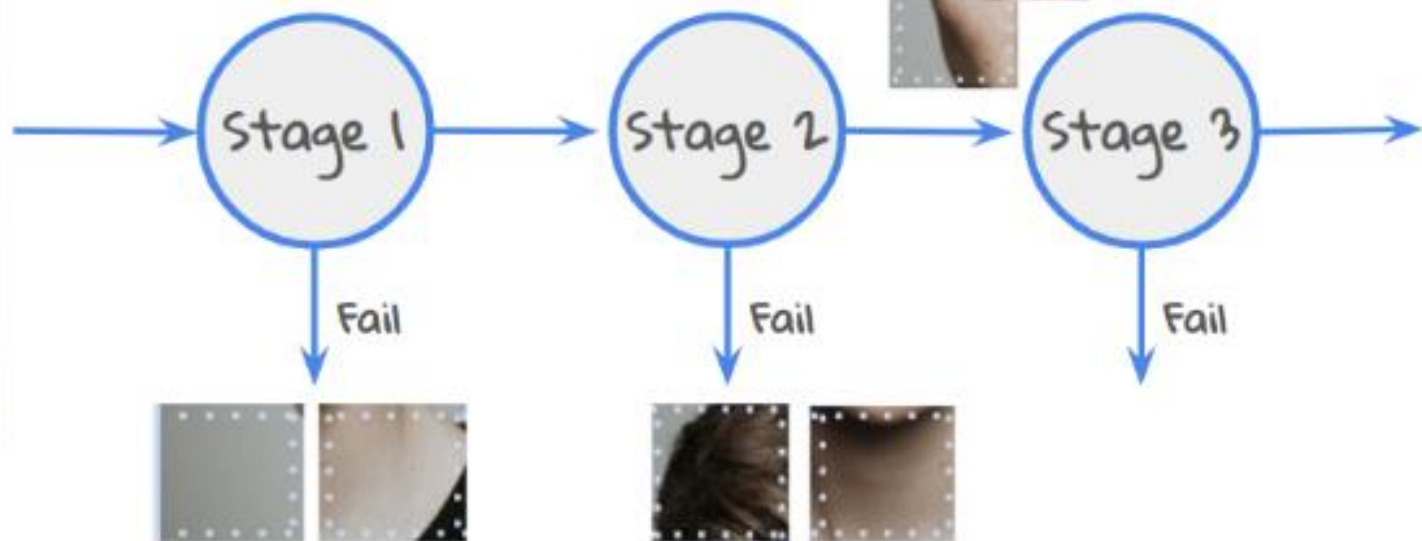


- Four-rectangle Features



0.6	0.8	0.5	0.6	0.6	0.7
0.6	0.5	0.7	0.7	0.8	0.8
0.1	0.1	0.1	0.2	0.3	0.2
0.2	0.3	0.2	0.3	0.2	0.1

→ Mean(dark region) - Mean(light region)



FACE!!

```

img = cv2.imread('captin_marvel.jpg')
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))

# Create the haar cascade
#cascPath = 'detect/haarcascade_frontalface_default.xml'
cascPath = 'haarcascades/haarcascade_frontalface_alt.xml'
faceCascade = cv2.CascadeClassifier(cascPath)

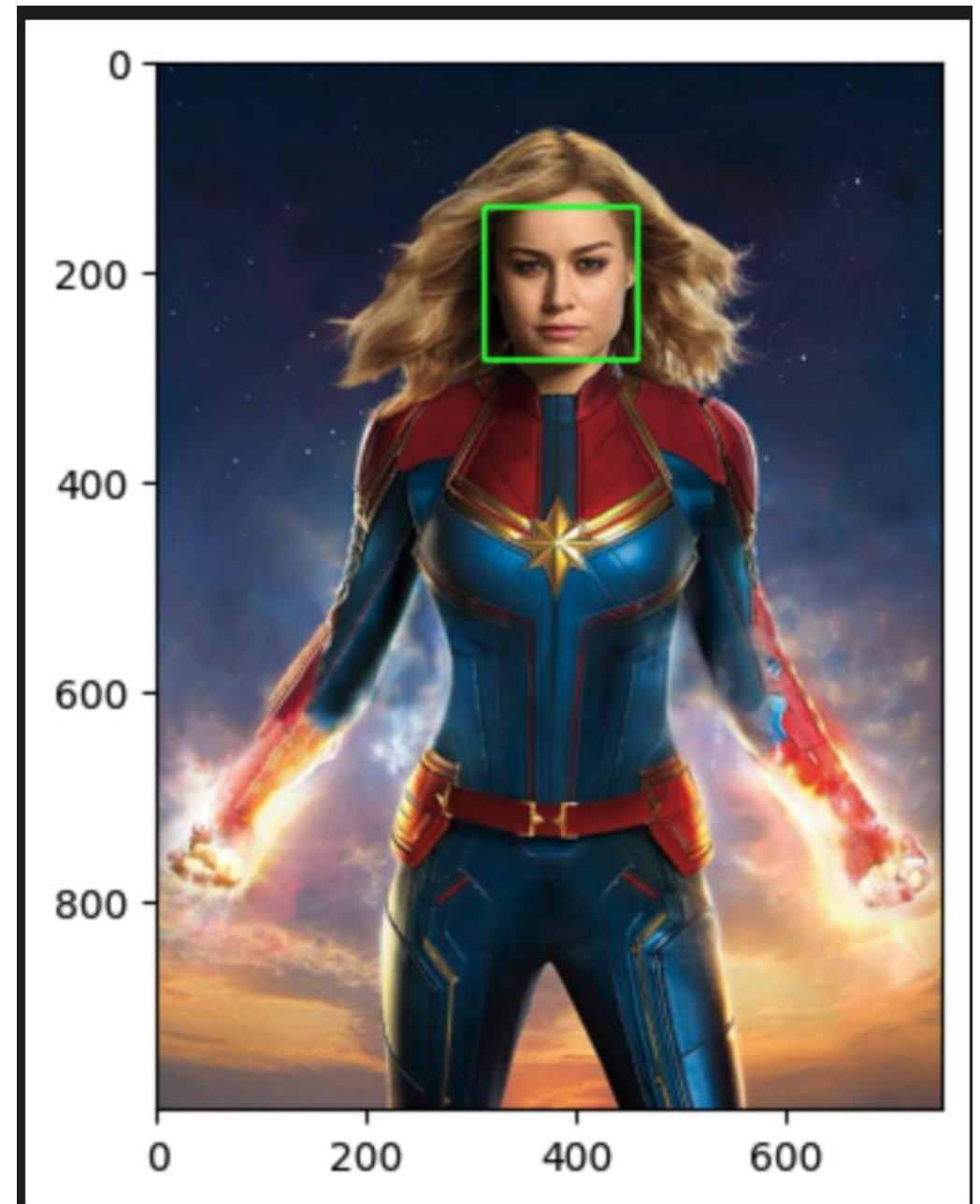
def detect_faces_show(fpath):
    img = cv2.imread(fpath)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # 檢測圖像中的面部
    # Detect faces in the image
    faces = faceCascade.detectMultiScale(
        img_rgb,
        scaleFactor=1.2,
        minNeighbors=5,
        minSize=(30, 30),
        flags = cv2.CASCADE_SCALE_IMAGE
    )
    print("Found %d faces!" % len(faces))

    # 臉部周圍畫一個矩形
    # Draw a rectangle around the faces

    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 4)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
detect_faces_show('captin_marvel.jpg')

```



```
import cv2
import numpy as np

face_cascade = cv2.CascadeClassifier('haarcascades/haarcascade_frontalface_default.xml')

def detect_face(img):
    img_copy = img.copy()
    face_rects = face_cascade.detectMultiScale(img_copy)

    for (x, y, w, h) in face_rects:
        cv2.rectangle(img_copy, (x, y), (x+w, y+h), (0, 255, 0), 3)

    return img_copy

cap = cv2.VideoCapture(1)

while True:
    ret, frame = cap.read(1)
    frame = detect_face(frame)
    cv2.imshow('Video Face Detection', frame)
    c = cv2.waitKey(1)
    if c == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

Images Contour Detection

- Contour Line: boundaries with the same value or intensity.
- Contour map/contour plot
- Edge v.s. Contour line
- Edge: in local range
- Contour: overall boundary of a figure

cv2.findContours()

- Parameters:**
- **image** – Source, an 8-bit single-channel image. Non-zero pixels are treated as 1's. Zero pixels remain 0's, so the image is treated as `binary`. You can use `compare()`, `inRange()`, `threshold()`, `adaptiveThreshold()`, `Canny()`, and others to create a binary image out of a grayscale or color one. The function modifies the `image` while extracting the contours. If mode equals to `CV_RETR_CCOMP` or `CV_RETR_FLOODFILL`, the input can also be a 32-bit integer image of labels (`cv_32sc1`).
 - **contours** – Detected contours. Each contour is stored as a vector of points.
 - **hierarchy** – Optional output vector, containing information about the image topology. It has as many elements as the number of contours. For each *i*-th contour `contours[i]`, the elements `hierarchy[i][0]`, `hierarchy[i][1]`, `hierarchy[i][2]`, and `hierarchy[i][3]` are set to 0-based indices in `contours` of the next and previous contours at the same hierarchical level, the first child contour and the parent contour, respectively. If for the contour *i* there are no next, previous, parent, or nested contours, the corresponding elements of `hierarchy[i]` will be negative.
 - **mode** –

Contour retrieval mode (if you use Python see also a note below).

- **CV_RETR_EXTERNAL** retrieves only the extreme outer contours. It sets `hierarchy[i][2]=hierarchy[i][3]=-1` for all the contours.
- **CV_RETR_LIST** retrieves all of the contours without establishing any hierarchical relationships.
- **CV_RETR_CCOMP** retrieves all of the contours and organizes them into a two-level hierarchy. At the top level, there are external boundaries of the components. At the second level, there are boundaries of the holes. If there is another contour inside a hole of a connected component, it is still put at the top level.
- **CV_RETR_TREE** retrieves all of the contours and reconstructs a full hierarchy of nested contours. This full hierarchy is built and shown in the OpenCV `contours.c` demo.
- **method** –

Contour approximation method (if you use Python see also a note below).

- **CV_CHAIN_APPROX_NONE** stores absolutely all the contour points. That is, any 2 subsequent points (x_1, y_1) and (x_2, y_2) of the contour will be either horizontal, vertical or diagonal neighbors, that is, $\max(\text{abs}(x_1 - x_2), \text{abs}(y_1 - y_2)) = 1$.
- **CV_CHAIN_APPROX_SIMPLE** compresses horizontal, vertical, and diagonal segments and leaves only their end points. For example, an up-right rectangular contour is encoded with 4 points.
- **CV_CHAIN_APPROX_TC89_L1, CV_CHAIN_APPROX_TC89_KCOS** applies one of the flavors of the Teh-Chin chain approximation algorithm. See [\[TehChin89\]](#) for details.
- **offset** – Optional offset by which every contour point is shifted. This is useful if the contours are extracted from the image ROI and then they should be analyzed in the whole image context.

https://docs.opencv.org/2.4/modules/imgproc/doc/structural_analysis_and_shape_descriptors.html#findContours

cv2.findContours()

参数:

- image: 8位单通道图像。非零像素值视为1，所以图像视作二值图像
- mode: 轮廓检索的方式
 - cv2.RETR_EXTERNAL: 只检索外部轮廓
 - cv2.RETR_LIST: 检测所有轮廓且不建立层次结构
 - cv2.RETR_CCOMP: 检测所有轮廓，建立两级层次结构 → 上面的一層為外邊界，裡面的一層為內孔的邊界資訊。如果內孔內還有一個連通物體，這個物體的邊界也在頂層。
 - cv2.RETR_TREE: 检测所有轮廓，建立完整的层次结构
- method: 轮廓近似的方法
 - cv2.CHAIN_APPROX_NONE: 存储所有的轮廓点
 - cv2.CHAIN_APPROX_SIMPLE: 压缩水平，垂直和对角线段，只留下端点。例如矩形轮廓可以用4个点编码。
 - cv2.CHAIN_APPROX_TC89_L1, cv2.CHAIN_APPROX_TC89_KCOS: 使用Teh-Chini chain近似算法
- offset: (可选参数) 轮廓点的偏移量，格式为tuple, 如 (-10, 10) 表示轮廓点沿X负方向偏移10个像素点，沿Y正方向偏移10个像素点

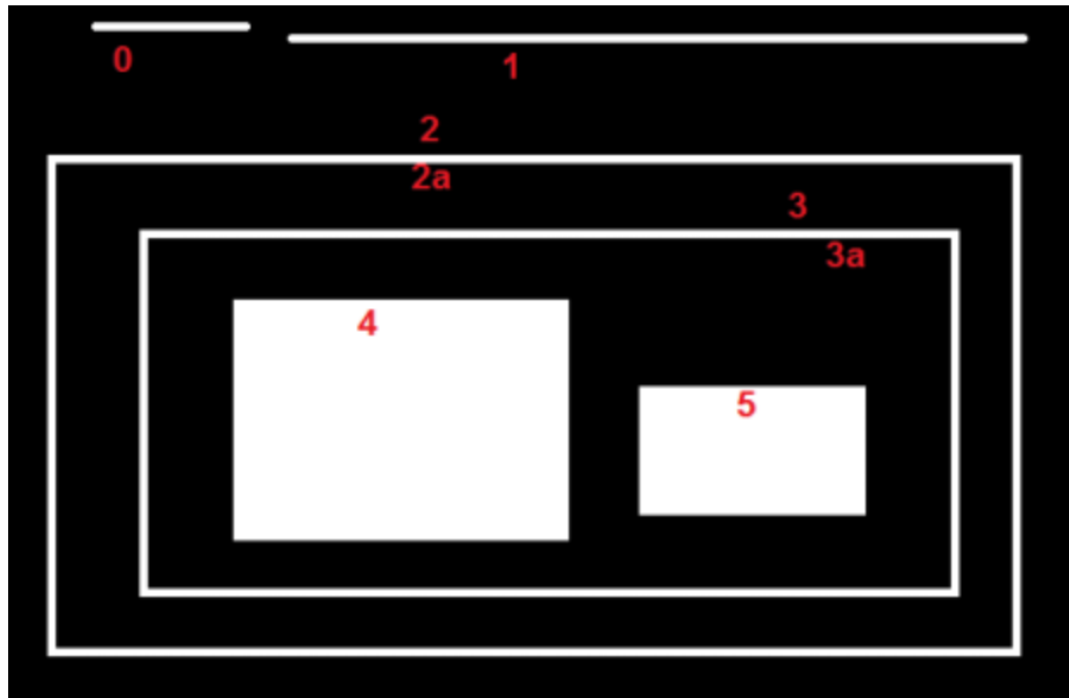
```
1.print (type(contours))  
2.print (type(contours[0]))  
3.print (len(contours))
```

https://blog.csdn.net/weixin_42216109/article/details/89840323

What is Hierarchy?

Normally we use the `cv2.findContours()` function to detect objects in an image, right ? Sometimes objects are in different locations. But in some cases, some shapes are inside other shapes. Just like nested figures. In this case, we call outer one as **parent** and inner one as **child**. This way, contours in an image has some relationship to each other. And we can specify how one contour is connected to each other, like, is it child of some other contour, or is it a parent etc. Representation of this relationship is called the **Hierarchy**.

Consider an example image below :



In this image, there are a few shapes which I have numbered from 0-5. *2 and 2a* denotes the external and internal contours of the outermost box.

Here, contours 0,1,2 are **external or outermost**. We can say, they are in **hierarchy-0** or simply they are in **same hierarchy level**.

Cv2.drawContours()

- image:需要绘制轮廓的目标图像，注意会改变原图
- contours:轮廓点，上述函数cv2.findContours()的第一个返回值
- contourIdx:轮廓的索引，表示绘制第几个轮廓，-1表示绘制所有的轮廓
- color:绘制轮廓的颜色
- thickness:（可选参数）轮廓线的宽度，-1表示填充
- lineType:（可选参数）轮廓线型，包括cv2.LINE_4,cv2.LINE_8（默认）,cv2.LINE_AA,分别表示4邻域线，8邻域线，抗锯齿线（可以更好地显示曲线）
- hierarchy:（可选参数）层级结构，上述函数cv2.findContours()的第二个返回值，配合maxLevel参数使用
- maxLevel:（可选参数）等于0表示只绘制指定的轮廓，等于1表示绘制指定轮廓及其下一级子轮廓，等于2表示绘制指定轮廓及其所有子轮廓
- offset:（可选参数）轮廓点的偏移量

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

def display(img, cmap = None):
    fig = plt.figure(figsize=(10,8))
    ax = fig.add_subplot(111)
    ax.imshow(img, cmap = cmap)

img = cv2.imread('pine_apple.jpg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
display(img)

# Blurring for removing the noise
img_blur = cv2.bilateralFilter(img, d = 7,
                               sigmaSpace = 75, sigmaColor = 75)
display(img_blur)
#Convert to grayscale
img_gray = cv2.cvtColor(img_blur, cv2.COLOR_RGB2GRAY)
#Apply the thresholding
a = img_gray.max()
_, thresh = cv2.threshold(img_gray, a/2+60, a, cv2.THRESH_BINARY_INV)
display(thresh, cmap = 'gray')
#find contours
contours, hierarchy = cv2.findContours(
    image = thresh,
    mode = cv2.RETR_TREE,
    method = cv2.CHAIN_APPROX_SIMPLE)

cv2.drawContours(img, contours, -1, (0,0,255), 3)
display(img, cmap = 'gray')

```



Contours

- Image moment:
 - help to find the centroid of an image or calculate the area of a boundary field.
- Image moments:
 - Image moments are weighted averages of pixel intensities based on their spatial distribution. Centroids or spatial information can be obtained from image moments.

- **Spatial Moments**

$$M_{ij} = \sum_x \sum_y x^i y^j I(x, y) \quad \rightarrow \quad m00, m01, m02, m03, m10, m11, m12, m20, m21, m30$$

- **Central Moments**

$$\mu_{pq} = \sum_x \sum_y (x - \bar{x})^p (y - \bar{y})^q I(x, y) \quad \rightarrow \quad \mu02, \mu03, \mu11, \mu12, \mu20, \mu21, \mu30$$

$$\text{- Centroid point : } \bar{x} = \frac{M_{10}}{M_{00}}, \quad \bar{y} = \frac{M_{01}}{M_{00}}$$

- **Normalized Central Moments**

$$\eta_{ij} = \frac{\mu_{ij}}{\mu_{00}^{(1+i+j)/2}} \quad \rightarrow \quad \eta02, \eta03, \eta11, \eta12, \eta20, \eta21, \eta30$$

Hu Invariant Moments

- <https://ithelp.ithome.com.tw/articles/10329086>

$$hu_1 = v_{20} + v_{02}$$

$$hu_2 = (v_{20} - v_{02})^2 + 4v_{11}^2$$

$$hu_3 = (v_{30} - 3v_{12})^2 + (3v_{21} - v_{03})^2$$

$$hu_4 = (v_{30} + v_{12})^2 + (v_{21} + v_{03})^2$$

$$hu_5 = (v_{30} - 3v_{12})(v_{30} + v_{12})[(v_{30} + v_{12})^2 - 3(v_{21} + v_{03})^2] + (3v_{21} - v_{03})(v_{21} + v_{03})[3(v_{30} + v_{12})^2 - (v_{21} + v_{03})^2]$$

$$hu_6 = (v_{20} - v_{02})[(v_{30} + v_{12})^2 - (v_{21} + v_{03})^2] + 4v_{11}(v_{30} + v_{12})(v_{21} + v_{03})$$

$$hu_7 = (3v_{21} - v_{03})(v_{30} + v_{12})[(v_{30} + v_{12})^2 - 3(v_{21} + v_{03})^2] - (v_{30} - 3v_{12})(v_{21} + v_{03})[3(v_{30} + v_{12})^2 - (v_{21} + v_{03})^2]$$

Misc

- `cv2.contourArea(contours[0]))`
- `cv2.arcLength(contours[0], closed = True)`
- `cv2.moments(contours[0])`

```

for i in range(len(contours)):
    # print(str(i) + "th Contour Area : ", cv2.contourArea(contours[i]))
c_4 = contours[4]
M = cv2.moments(c_4)
print(M.keys())
print(len(c_4))

#The centroid point
cx = int(M['m10'] / M['m00'])
cy = int(M['m01'] / M['m00'])

# The extreme points
print(c_4.shape)
l_m = tuple(c_4[c_4[:, :, 0].argmin()][0])
r_m = tuple(c_4[c_4[:, :, 0].argmax()][0])
t_m = tuple(c_4[c_4[:, :, 1].argmin()][0])
b_m = tuple(c_4[c_4[:, :, 1].argmax()][0])
pst = [l_m, r_m, t_m, b_m]
xcor = [p[0] for p in pst]
ycor = [p[1] for p in pst]

plt.figure(figsize = (10, 16))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap = 'gray')
plt.scatter([cx], [cy], c = 'g', s = 50)
plt.subplot(1, 2, 2)
plt.imshow(img, cmap = 'gray') # image
plt.scatter(xcor, ycor, c = 'r', s = 50)

```

